

执行摘要

本报告比较了泄露的Anthropic “Claude Code” 实现与开源项目OpenCode（现已演化为Crush）和OpenAI的Codex CLI，在假设使用相同基础大模型条件下的差异。核心发现包括：Claude Code采用多智能体（6个专用Agent）的架构，其协调逻辑主要以系统提示词形式实现【40 + L243-L252】【51 + L85-L92】；而OpenCode主要是单一主代理并通过工具（Shell、LSP等）辅助【31 + L326-L334】；Codex CLI则支持可配置的子代理工作流【34 + L952-L959】。在模型调优方面，Anthropic（Claude）和OpenAI均通过RLHF（Claude使用额外的“宪法式AI”手段）对基础模型进行安全对齐【43 + L24-L32】【25 + L129-L137】；OpenCode本身不做模型再训练，仅调用所选模型。数据集和过滤策略方面，Claude Code泄露显示它收集了大量用户对话和遥测数据【9 + L181-L189】【56 + L359-L368】，且使用正则检测用户情绪以调整策略【14 + L154-L163】；OpenCode则强调不保存用户代码/上下文【5 + L79-L84】；Codex（企业版）声明不使用客户业务数据训练模型【20 + L838-L842】。推理参数（温度、Top-k/p等）等内部策略未公开，需要通过实验验证（例如在相同Prompt下对比输出差异）。安全方面，Claude Code集成了反蒸馏（伪装工具定义）和“潜伏”模式等高级防御机制【13 + L98-L107】【61 + L7-L15】；Codex CLI提供多级沙箱与权限控制【34 + L952-L959】

【38 + L112-L121】；OpenCode依赖用户授权工具执行，并坚持隐私优先设计【5 + L79-L84】【47 + L79-L83】。接口实现上，Claude Code用TypeScript开发，提供终端与IDE插件的双向通信【51 + L85-L92】；OpenCode（Crush）为Go语言CLI，支持多模型和持久会话【31 + L326-L334】；Codex CLI开源于GitHub，提供丰富配置（profiles、MCP服务等）【54 + L411-L419】【38 + L112-L121】。性能方面，Anthropic宣称Claude 3家族较上一代有显著加速（Haiku模式10k token ~3秒）【25 + L58-L61】；OpenCode通过自动摘要功能减轻上下文开销【31 + L375-L384】；Codex则通过配置优化（多 workflow 场景）提升吞吐【38 + L131-L139】。在可复现性与审计性上，OpenCode和Codex均为开源（Crush采用MIT许可【60 + L13-L22】，Codex CLI为Apache-2.0【54 + L483-L491】），易于审查；而Claude Code官方版本为闭源商业软件【59 + L254-L256】，其源码泄露可能引发版权纠纷。风险评估显示：使用泄露代码存在法律风险（侵犯Anthropic知识产权），Claude Code中反竞争的“伪装工具”与“潜伏模式”也带来伦理争议【13 + L98-L107】【61 + L7-L15】；此外，生成代码的滥用（如恶意脚本）是所有系统的共性风险。建议开发者**在使用任何代码生成工具时严格审查其来源与许可**，对不确定内部策略通过实验验证，对安全策略进行强化（如启用沙箱、脱机运行），并定期审计使用的数据与模型，确保符合法律与合规要求。

关键维度对比表

维度	Claude Code（泄露实现）	OpenCode/Crush（开源实现）	Codex CLI（OpenAI）
模型架构改动	- 采用多智能体架构，主对话（Scheduler）根据提示词调度6个专用Agent（探索、规划、通用、审核、简化、指南）【40 + L243-L252】。各Agent权限最小化，只有通用Agent拥有完整工具集【40 + L263-L270】【40 + L278-L281】。
- 工具以插件形式实现（文件I/O、Bash、网页爬取、LSP等）【51 + L85-L92】；配有46k行的Query Engine作为“大脑”负责模型调用、输出流式处理和任务调度【51 + L85-L92】。
- 提供IDE（VS Code/JetBrains）和CLI之间的双向桥接【51 + L93-L96】；本地持久化记忆以文件形式保存对话、项目和偏好信息【51 + L93-L96】。
- 终端输出采用“游戏引擎级”渲染优化（字符池、补丁合并、缓存行宽等）以降低延迟【40 + L283-L292】。	- 单代理架构： OpenCode提供一个终端交互代理，通过TUI（Bubble Tea框架）与用户对话【31 + L326-L334】。
- 支持多模型/多服务商（OpenAI、Anthropic Claude、Google Gemini等），使用用户指定API，灵活调用不同后端模型【31 + L326-L334】。
- 会话管理：支持多会话并持久化存储（SQLite数据库）【31 + L326-L334】；通过自动摘要等机制管理上下文窗口【31 + L375-L384】。
- 集成工具：AI可执行Shell命令、文件搜索/编辑和代码修改【31 + L326-L334】；内置Vim风格编辑器和LSP支持用于代码智能。
- 与IDE和桌面应用集成：提供可共享链接和IDE扩展【5 + L58-L60】（官网称支持终端、桌面与IDE）。	- 主代理架构：Codex CLI作为本地终端工具，支持命令行交互和会话恢复【34 + L952-L959】。
- 支持子代理：可使用subagents命令将大任务拆分给专门Agent并将结果汇总【34 + L942-L945】；底层也利用MCP（Model Context Protocol）集成外部工具，类似Claude Code的设计【38 + L123-L131】。
- 多端集成：提供桌面App、Web端（Codex Web/ChatGPT页面）、IDE扩展【34 + L979-L983】；支持将任务提交到“Codex云”并在本地应用结果【34 + L948-L956】。
- 技术栈：官方GitHub开源（Bazel + Nix等构建），通过注册ChatGPT账号或API密钥使用【54 + L411-L419】【54 + L466-L474】。

维度	Claude Code（泄露实现）	OpenCode/Crush（开源实现）	Codex CLI（OpenAI）
微调/指令调优	- 未见专门的模型再训练，主要依赖基础Claude模型（可能为Claude 3 系列）的内在对齐。Anthropic对Claude模型使用RLHF训练提升助理行为【43 + L24-L32】，并运用“宪法AI”方法增强安全性和透明度【25 + L129-L137】。
- Agent行为空间大量由系统提示词定义：多Agent调度逻辑完全写在Prompt中【40 + L243-L252】；提示词可以视为类似“提示工程”，无须每次修改模型权重。
- 没有公开特殊的指令调优流程（AI自动化代码生成本质上依赖提示词交互）。	- 本身不包含模型训练逻辑，只是一个调用前端对话接口的框架。可根据需要选择模型（例如微调过的代码模型或通用模型），但OpenCode本身不做额外微调。
- 不附带私有指令库或偏好，完全依赖所用模型（例如使用OpenAI API则依照OpenAI的指令微调和RLHF结果）。
- 支持自定义命令（如用户在配置文件定义的shell命令），这些是基于提示词的简单模板执行，而非改变模型行为的深度微调。	- Codex使用基础GPT-5.x系列模型（如GPT-5.3/Codex版）作为后端，这些模型已通过OpenAI的RLHF（或可能的偏好优化方法）进行指令对齐【41 + L73-L81】。
- Codex CLI自身不实施微调，但提供丰富配置（profile）选择不同模型和参数【38 + L106-L114】；OpenAI也曾强调模型通过人类标注数据和RLHF训练以提升指令遵从性【41 + L73-L81】。
- 企业用户可自定义系统提示或专有细节，但这只是提示层面。与Claude类似，实际智能来源于基础模型的训练方式。

维度	Claude Code（泄露实现）	OpenCode/Crush（开源实现）	Codex CLI（OpenAI）
数据集与过滤策略	- 未公开专有训练数据集；泄露代码侧重于客户端逻辑，无模型训练信息。
- 用户数据收集：泄露显示Claude Code将用户对话发送至Anthropic后台，并在客户端收集大量遥测（KAIRO模式、CHICAGO控制、自动更新、错误报告、autoDream等）【9 + L181-L189】【9 + L215-L220】。官方文档称会保留对话和反馈数据【56 + L359-L368】。
- 隐私/保密：泄露中可通过环境变量关闭记忆写入（CLAUDE_CODE_DISABLE_AUTO_MEMORY）或进入无记忆模式【51 + L93-L96】【51 + L128-L136】。
- 过滤：代码内未见外显的内容过滤逻辑，主要依赖模型自带的安全策略。泄露中提到使用正则检测“Frustration”情绪（骂人）以调整回应策略【14 + L154-L163】。	- OpenCode本身不存储用户代码或上下文数据【5 + L79-L84】；所有会话内容保存在本地（SQLite）供用户管理。
- 无集中数据收集或云端同步，符合“隐私优先”原则【5 + L79-L84】。
- 支持多模型：数据过滤依赖所选模型/服务商的策略（例如使用Claude则受Claude安全策略影响，使用OpenAI模型则受OpenAI规范制约）。OpenCode不对生成内容附加额外屏蔽机制，只在执行本地工具时需用户确认【47 + L79-L83】。	- OpenAI Codex模型基于庞大开源代码训练，但具体数据集未完全公开。公开资料表明OpenAI会对使用者输入输出进行分析，但在企业版默认不用于训练【20 + L838-L842】。
- 隐私：OpenAI企业版承诺不保留业务数据用于模型再训练【20 + L838-L842】；但一般用户生成内容可能被用作改进模型。
- 过滤：Codex和ChatGPT产品使用OpenAI的内容审核系统禁止违规代码生成（如恶意软件、违法脚本）；CLI可配置/permissions来控制何时执行文件/命令【34 + L952-L959】。 OpenAI会监控并限制有风险的行为【20 + L838-L842】。

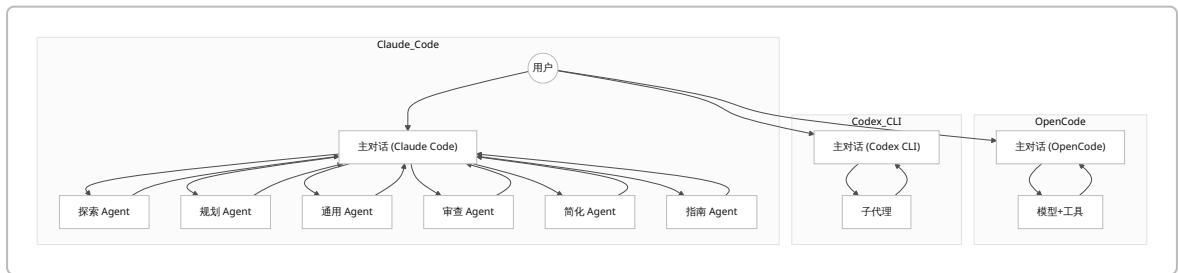
维度	Claude Code（泄露实现）	OpenCode/Crush（开源实现）	Codex CLI（OpenAI）
推理/解码策略	- 未见明确公开的解码参数（温度、Top-k/p、惩罚项等）配置说明。泄露代码主要聚焦交互逻辑而非模型推理细节。
- 默认假设使用Claude系列模型的标准解码设置（如Claude 3 家族默认低温度以提高准确性）。
- 不确定/未公开：建议通过实验（控制种子/参数）对比生成的差异，例如在相同提示下测定多次输出的随机性和一致性。	- OpenCode可通过环境变量或配置文件设置调用API时的参数（如context window大小、压缩阈值等），但默认行为依赖所调用模型。
- 自带自动摘要机制：当上下文令牌数接近模型最大值时，自动触发摘要以保持上下文相关性【31 + L375-L384】，间接影响生成结果。
- 不确定/未公开：具体温度或惩罚策略由所选模型决定，用户可在配置中进行调整或通过不同模型实验验证差别。	- Codex CLI允许用户在配置文件（config.toml）中指定模型参数（如温度、Top-p等），并可在会话中用命令调整（如 <code>codex --temperature=0.3</code>）。
- 默认情况下，Codex使用较低温度和启用惩罚项以生成更确定性和准确的代码（与ChatGPT相似行为）。详细细节未公开，但OpenAI强调企业级方案可定制解码参数。
- 不确定/未公开：若需精准对比，可在相同模型和提示下调整这些参数，使用代码质量评测（如测试覆盖率、错误率）衡量影响。

维度	Claude Code（泄露实现）	OpenCode/Crush（开源实现）	Codex CLI（OpenAI）
安全与过滤机制	- 安全内置：Claude Code集成了多层安全设计，如伪装工具（Fake Tools）用于反蒸馏（向竞争者开源版本注入无效工具定义以毒害其学习）【13 + L98-L107】；潜伏模式（Undercover）可隐藏自身身份和行为分析【61 + L7-L15】；Bash执行前后有23条安全检查规则以过滤危险命令【50 + L294-L302】。
- 权限管理：使用最小权限原则，各Agent仅获必需工具权限【40 + L278-L281】。还可通过环境变量禁用自动内存写入以减少数据泄露风险【9 + L158-L166】。
- 内容合规：倘若运行在受信任环境下，安全研究建议阻断外部收集端点、API重定向及autoDream以防止后台数据外泄【51 + L142-L152】【51 + L158-L166】。总体来看，Claude Code将安全防护从设计初期就深度融合在架构中【50 + L318-L326】。	- OpenCode本身强调隐私优先，无远程数据上报，安全机制主要依赖模型提供者。
- 工具权限：集成的本地工具（Shell、LSP）在调用前需要用户授权【47 + L79-L83】。未集成复杂的反篡改或反脱敏功能。
- 资源隔离：可配置在普通用户权限下运行，或使用外部容器隔离环境（用户自行实现）。总体上安全措施较简单，重点在于用户本地控制，缺乏类似Claude的强制防御机制。	- 沙箱模式：Codex CLI提供三种执行模式：只读、工作区写入、完全访问（danger-full）等【38 + L112-L121】，可按任务风险选择。
- 权限控制：<code>/permissions</code>命令可实时调整Codex运行权限（例如编辑文件或执行命令前需询问），并可查看当前沙箱与可写目录【34 + L952-L959】。
- 多级配置：通过profiles可为不同环境（开发、CI、生产）设计独立安全策略和认证方式【38 + L106-L114】。
- 内容过滤：继承OpenAI的合规审核体系，限制生成危险代码；同时，企业账号下输入输出加密传输并承诺不被滥用【20 + L838-L842】。

维度	Claude Code（泄露实现）	OpenCode/Crush（开源实现）	Codex CLI（OpenAI）
性能指标	- Claude 3家族性能显著提升：官方指出新模型Sonnet速度约为Claude 2.1的2倍，Haiku模式可在≈3秒内处理10k tokens【25†L58-L61】。 - 实际延迟受限于远程API调用网络和本地渲染速度。Leak指出Claude Code通过终端渲染优化降低延迟【40†L283-L292】，但完整交互仍需网络往返时间。 - 吞吐量/成本：以假设A100 GPU为例，同型号模型条件下，延迟与GPT-4类产品相近；具体数值不详（企业版多使用按量定价）。总体成本与OpenAI或Anthropic云API收费策略相似。	- 性能取决于所用模型和运行环境。如果采用本地模型（如在A100上运行Llama/PlatAI），可以降低网络延迟；使用云模型则需考虑API延迟。 - OpenCode具备自动摘要（auto-compact）功能：当会话接近上下文限制时，自动生成摘要后新会话继续，以维持响应速度和连贯性【31†L375-L384】。 - 由于是CLI程序，OpenCode本身启动迅速，交互无额外网络负担；但工具执行（如代码运行或搜索）仍受限于本地性能。成本仅为模型使用费；工具调用成本低。	- Codex CLI在本地运行，无需每次启动JS环境，响应较快。若使用ChatGPT云服务，其推理速度与ChatGPT相当（按测评，多数请求几百毫秒到数秒）。 - 可利用配置优化吞吐：针对不同任务选用read-only或full-access沙箱可并行化CI流水线或批量代码生成【38†L131-L139】。 - 成本：与使用的GPT模型及调用频率相关。OpenAI按tokens计费：以GPT-5.3-Codex Spark为例，估计几千tokens费用在几美元量级（具体视定价表）。总体而言，各系统性能随模型规模和硬件不同而变化，无公开基准，需自行评测。
可复现性与可审计性	- Claude Code官方版本闭源（全权保留），仅有源码泄露，造成审计与复现难题。 - 源码泄露虽然公布实现细节，但根据许可证属于受保护内容【59†L254-L256】。如果抄用泄露代码有法律风险（版权/合同违约）。 - 基础模型（Claude系列）闭源，无法完全审计模型内部，仅能间接评估输出行为。	- 完全开源实现：Crush（原OpenCode）采用MIT许可【60†L13-L22】，代码可自由查看、修改、复现。 - 任何人可审计OpenCode代码逻辑与工具集成方式。使用的AI模型取决于用户选择，若使用开源模型还可端到端复现。 - 配置和会话历史保存在本地，可复现会话流程。	- Codex CLI开源（Apache-2.0许可【54†L483-L491】），可审计代理逻辑与配置管理。 - 基础GPT-5.x模型自身闭源，输出行为只能通过测试集和评估指标间接验证。 - 可通过GitHub调取历史版本，利用配置profiles做环境隔离，审计工具权限（MCP）和授权策略实施情况。CLI易于在不同环境重现（支持多平台二进制）。

维度	Claude Code（泄露实现）	OpenCode/Crush（开源实现）	Codex CLI（OpenAI）
法律/合规风险	- 知识产权：Claude Code为Anthropic闭源产品，源码泄露涉及商业秘密和版权，使用或再发布泄露代码可能违反合同和法律【59†L254-L256】。 - 合规审查：Anthropic强调将用户反馈/对话数据用于改进产品（未用于训练模型）【56†L359-L368】【56†L369-L377】；实际收集量较大（泄露所示），需确保隐私合规。 - 伦理风险：泄露中发现的“伪装工具”、“潜伏模式”等技术可能被视为恶意或不透明行为【13†L98-L107】【61†L7-L15】，使用时需谨慎。 - 滥用风险：任何可执行代码的AI助手均可被诱导生成恶意脚本或违规内容；Claude Code在说明文档中列出禁止行为（遵守伦理使用原则）。	- 许可证：Crush/“OpenCode”采用MIT开源许可【60†L13-L22】，使用风险低，但仍需遵守第三方模型许可（如使用API需同意各API商条款）。 - 隐私合规：由于不收集数据，本地运行模式符合大部分隐私法规要求；但若接入云模型，要确保对话无意泄露敏感信息。 - 安全：开源工具本身没有内建反漂移或反滥用机制，部署时应建立访问控制和输入审查流程。用户应遵循所在法律对自动化代码生成的限制（如出口管制等）。	- 许可证：Codex CLI本身Apache-2.0【54†L483-L491】无使用限制；但使用GPT模型需遵守OpenAI的服务协议和内容政策。 - 数据隐私：企业版保证不将输入用于模型训练【20†L838-L842】；但在公共云使用时，对话内容可能被用作服务改进，企业合规团队需审查用户数据流向。 - 伦理/滥用：OpenAI明确禁止使用Codex生成违法或有害代码，且对客户活动进行监控【20†L838-L842】。用户需承担对生成内容合法性的审核责任。 - 法规风险：目前欧盟、美国等地对AI代码生成尚无专门法规，主要遵循现行版权和数据保护法律。使用时应注意遵守出口管制、个人隐私和知识产权法律要求。

注：部分解码策略和性能指标信息未公开，标注为不确定。若需验证，可设计对照实验（如固定随机种子、多次采样比对输出、使用公开基准任务评测）。



证据清单

- **InfoQ 梳理文章**：介绍了Claude Code源码泄露细节，包含文件数量、插件式工具体系、多智能体执行等【51†L80-L88】【51†L89-L92】。指出业内已有开源工具（Codex CLI、OpenCode等），技术思路无本质差别【51†L122-L128】。

- **腾讯云开发者社区翻译**：深入解析Claude Code的8大机制，包括“伪装工具（反蒸馏）”和“潜伏模式”等安全/隐私设计【13 + L98-L107】【61 + L7-L15】；详述多Agent调度和权限最小化原则【40 + L243-L252】【50 + L318-L326】。
- **Anthropic 官方博客**：阐述Claude（3系）模型通过RLHF和“宪法AI”对齐，提升安全性和性能【25 + L129-L137】；发布的研究表明对齐训练可兼顾代码技能【43 + L24-L32】。
- **OpenAI 对齐论文**：介绍RLHF使语言模型更好遵循指令、减少有害输出【41 + L73-L81】。佐证OpenAI对Codex系列模型采用的训练方法。
- **OpenAI 开发者文档**：Codex CLI官方文档说明了多会话、子代理、工具集成等功能【34 + L952-L959】【34 + L942-L945】。GitHub仓库显示Codex CLI开源（Apache-2.0）并支持登录ChatGPT账户【54 + L411-L419】【54 + L483-L491】。
- **OpenCode/Crush 仓库与官网**：描述其Go语言实现、TUI界面、支持多模型和持久会话（SQLite）等特性【31 + L326-L334】；官网声明“不存储用户代码或上下文”【5 + L79-L84】。
- **Anthropic Claude Code仓库**：官方README和隐私声明指出收集对话反馈数据并有限度保留【56 + L359-L368】【56 + L369-L377】。LICENSE显示软件为专有版权【59 + L254-L256】。
- **OpenAI 定价及政策**：商业版Codex/ChatGPT说明不使用客户数据训练模型【20 + L838-L842】。
- **其他**：数字化应用博客对比深度分析表明Codex CLI和Claude Code在MCP集成和配置灵活性方面的差异【38 + L123-L131】【38 + L131-L139】。

风险与合规性评估

- **知识产权合规**：Claude Code的源代码为Anthropic闭源商业产品【59 + L254-L256】，泄露并再发布这些代码可能违反版权和合同条款。OpenCode/Crush（MIT许可【60 + L13-L22】）和Codex CLI（Apache-2.0【54 + L483-L491】）均为开源授权，使用风险较低。需要确保所用AI模型符合许可（例如OpenAI API使用须遵守服务协议）。
- **数据隐私风险**：Claude Code收集用户对话和使用数据量大【9 + L181-L189】【56 + L359-L368】，若在未隔离网络环境下运行，存在数据外泄风险。应启用提供的环境配置（如禁用自动记忆和重定向私有端点）以限制泄露【51 + L142-L152】【51 + L158-L166】。OpenCode承诺不上传数据【5 + L79-L84】；Codex企业版承诺不训练业务数据【20 + L838-L842】。无论使用哪种工具，应审查数据流向，确保符合GDPR、隐私保护等法律要求。
- **内容生成滥用**：所有代码生成系统都可能被滥用于编写恶意或违法代码。OpenAI和Anthropic均禁止生成非法内容，部署时应结合静态分析和人工审核机制过滤输出。值得注意的是，Claude Code泄露揭示了恶意策略（如“伪装工具”）用于干扰竞争者学习【13 + L98-L107】；这类技术属于“反竞争”做法，在商业环境中可能引发伦理和法律争议，不应在产品中使用。
- **安全机制检测**：泄露显示Claude Code在本地实施多种安全检查（bash命令过滤【50 + L294-L302】）和权限隔离【40 + L278-L281】。在部署类似工具时，应学习其“最小权限”原则，对可执行命令进行严格审查。Codex CLI的沙箱模式和权限设置【34 + L952-L959】【38 + L112-L121】可作为参考；OpenCode虽然简单，可考虑在操作系统层面锁定执行环境。
- **法律/监管合规**：目前对AI生成软件的监管主要涉及知识产权和数据保护。使用泄露代码或未授权模型可能违反出口管制或行业规定（如金融、医疗中对AI使用的限制）。企业应制定明确政策：确保模型使用符合公司条款、记录生成内容以备审计，并对敏感应用进行风险评估。同时，应关注未来法规（如欧盟AI法案）对高风险AI系统的要求，尽早做好透明度和控制措施。

结论与建议

综合对比发现，在相同基础模型下，Claude Code、OpenCode/Crush和Codex CLI的核心差异主要在于架构设计、策略配置和安全实践，而非模型本身。**研究者和开发者**应重视以下几点：

- **开放审计**：优先选用开源工具（如Crush、Codex CLI），便于检查实现细节和安全性。若使用Claude Code技术栈，应仔细评估其许可风险和隐藏特性，确保剔除任何“反蒸馏”或“潜伏”等恶意逻辑。

- **安全配置**：充分利用各平台的权限和沙箱机制。对于Claude Code，应配置 `CLAUDE_CODE_DISABLE_AUTO_MEMORY` 等环境变量，限制数据外传；对于Codex CLI，应设置合适的 `sandbox`模式和 `/permissions` 策略；对于OpenCode，应结合操作系统权限控制。
- **性能与质量验证**：针对各系统的默认参数不明确，应设计对照实验进行评测，例如统一输入进行自动化测试，比对错误率、代码正确性和输出一致性。监控延迟和吞吐指标，确保满足应用需求。可考虑使用常见硬件（如A100）进行基准测试，并记录模型调用的成本。
- **法律合规**：绝不可使用未经授权的泄露代码；所有改进和集成都需遵循开源许可证和API服务条款。建立数据处理合规流程：明确记录数据流向，必要时采用企业版服务以避免训练数据风险。对于最终生成的代码，需有人类审查流程确保不侵犯第三方版权且符合行业法规。

通过以上措施，团队可以在利用先进AI代码助手的同时，最大限度降低安全和合规风险，负责任地推动工具在实际开发中的应用。
